

The Fourier Transform



LOGO

The logo is a circular emblem divided into four equal quadrants by a white border. The top quadrant shows a close-up of a yellow computer keyboard with a 'help' key. The right quadrant shows a blue and green grid pattern. The bottom quadrant shows a hand holding a pen over a document. The left quadrant shows a hand holding a pen over a document. In the center of the logo is a white circle containing the word 'LOGO' in bold, dark blue capital letters.

Why use Fourier Transform



- ❖ Perform impossible tasks to other ways .
- ❖ Perform tasks more quickly.
- ❖ Alternative to linear spatial filtering.
- ❖ Perform (low-high) pass filtering .

introduction



- ❖ Any function could be written in :
sum of sines and cosines of
varying amplitudes and frequencies
- ❖ It may require a finite or infinite number of
functions :
- ❖ ex "Square wave"

$$f(x) = \sin x + \frac{1}{3} \sin 3x + \frac{1}{5} \sin 5x + \frac{1}{7} \sin 7x + \frac{1}{9} \sin 9x + \dots$$

Ex “Square wave”

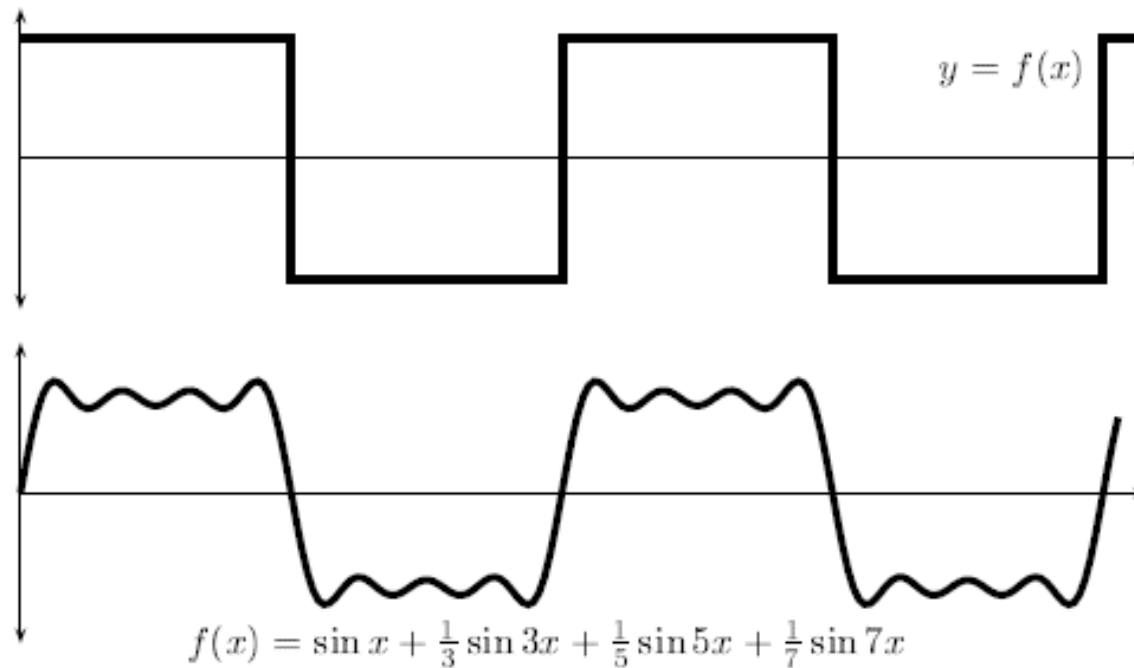


Figure 4.2: A square wave and its trigonometric approximation

Ex2 “function & its trigonometric decomposition”

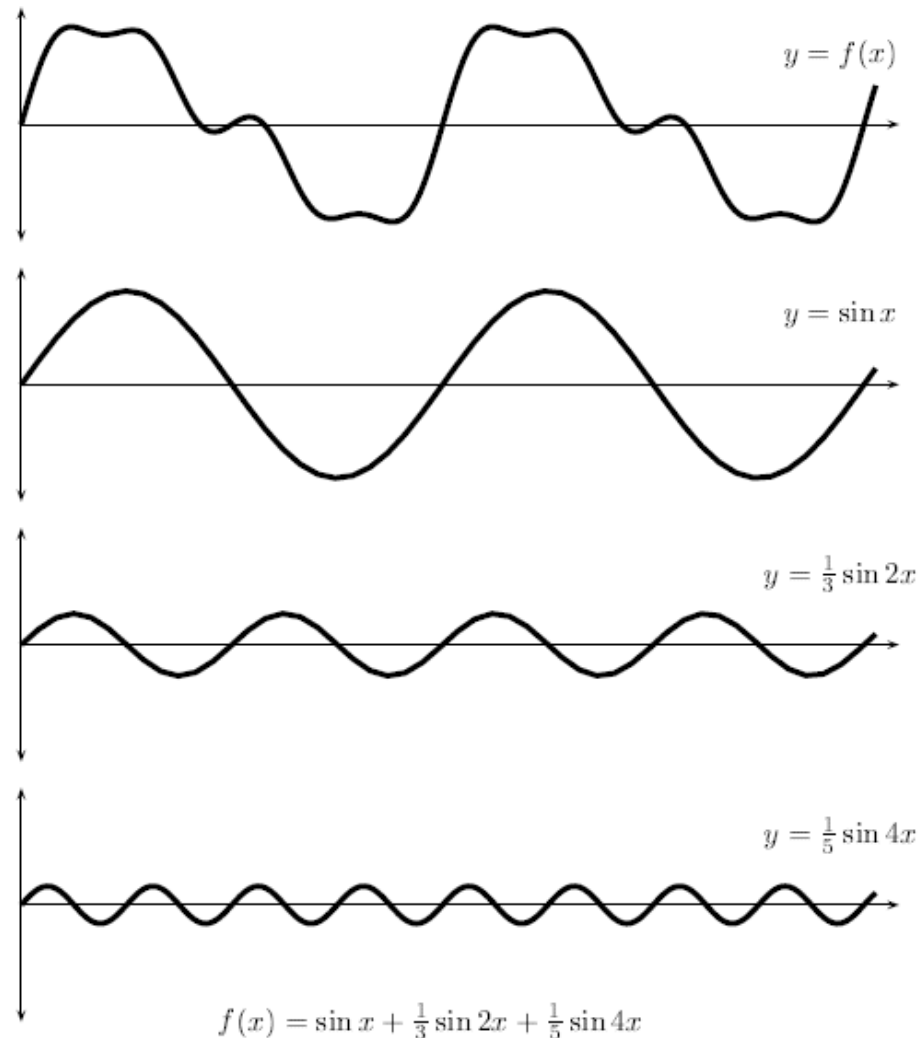


Figure 4.1: A function and its trigonometric decomposition

One dimensional DFT



- ❖ our concern in image processing with **discrete values** so we have to obtain finite number of functions .
- ❖ We can take discrete values (samples) of the wave as follow:

1, 1, 1, 1, -1, -1, -1, -1

One dimensional DFT

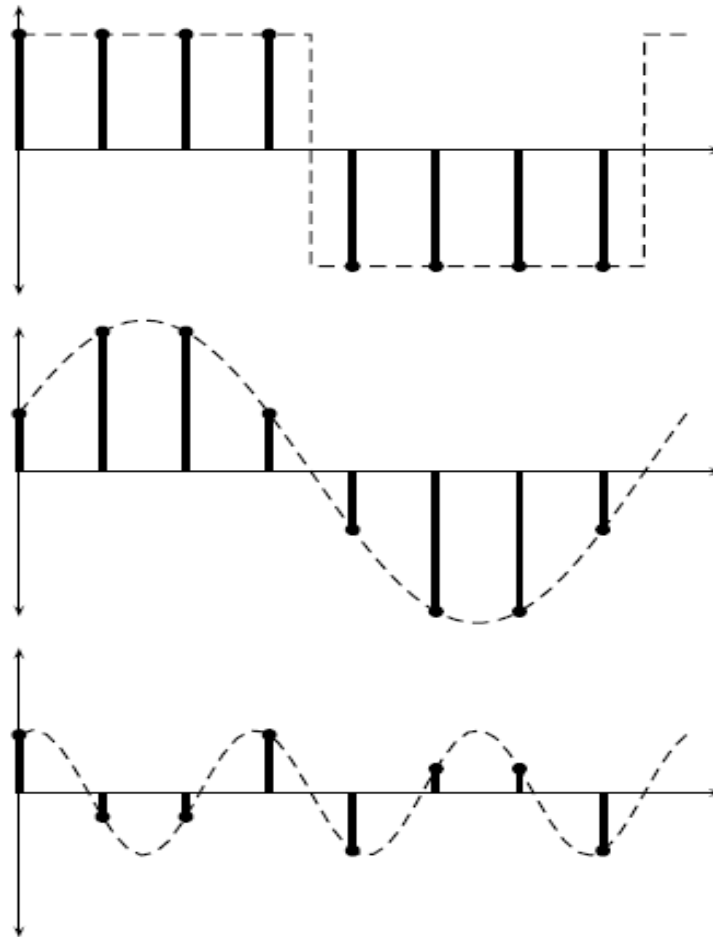


Figure 4.3: Expressing a discrete function as the sum of sines

One dimensional DFT



❖ Definitions :

- If we have the discrete sequence of length N :

$$\mathbf{f} = [f_0, f_1, f_2, \dots, f_{N-1}]$$

- Then we define its discrete Fourier Transform to be :

$$\mathbf{F} = [F_0, F_1, F_2, \dots, F_{N-1}]$$

- It will be calculated using the following formula:

$$F_u = \frac{1}{N} \sum_{x=0}^{N-1} \exp \left[-2\pi i \frac{xu}{N} \right] f_x.$$

- The Inverse DFT will be

$$x_u = \sum_{x=0}^{N-1} \exp \left[2\pi i \frac{xu}{N} \right] F_u.$$

- Comparison between forward & Inverse DFT :

1. there is no scaling factor $1/N$,
2. the sign inside the exponential function has been changed to positive.

The Fast Fourier Transform.



- ❖ The use of an FFT reduces the time needed to compute a DFT .
- ❖ FFT method works recursively by dividing the original vector into two halves, computing the FFT of each half, and then putting the results together .

The Fast Fourier Transform.



- ❖ For a vector of length N the direct method takes N^2 multiplication
- ❖ The FFT need $N \log_2 N$
- ❖ Ex. 1000 point
DFT needs $1000 \times 1000 = 1,000,000$
FFT needs $1024 \times 10 = 10,240$

The Fast Fourier Transform.



❖ **For a vector of length 2^n the direct method takes:** $(2^n)^2 = 2^{2n}$

❖ **The FFT need** $n2^n$

2^n	Direct arithmetic	FFT	Increase in speed
4	16	8	2.0
8	84	24	2.67
16	256	64	4.0
32	1024	160	6.4
64	4096	384	10.67
128	16384	896	18.3
256	65536	2048	32.0
512	262144	4608	56.9
1024	1048576	10240	102.4

Table 4.1: Comparison of FFT and direct arithmetic

The two-dimensional DFT



❖ In two dimensions DFT take a matrix .

- The original Matrix will be $f(x, y)$.

❖ The Fourier Transform of f will be

- $F(u, v)$

$$F = \mathcal{F}(f).$$

$$f = \mathcal{F}^{-1}(F).$$

❖ two-dimensional function can be expressed as sums of corrugation functions which have the general form :

$$z = a \sin(bx + cy).$$

The two-dimensional DFT



❖ **Fourier transform rewrites the original matrix in terms of sums of corrugations .**

❖ **The forward and inverse transforms for an $M \times N$ Matrix**

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) \exp \left[-2\pi i \left(\frac{xu}{M} + \frac{yv}{N} \right) \right] .$$

$$f(x, y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) \exp \left[2\pi i \left(\frac{xu}{M} + \frac{yv}{N} \right) \right] .$$

Properties OF 2D – Fourier Transform



- ❖ Similarity
- ❖ Seprability
- ❖ Linearity
- ❖ The Convolution Method
- ❖ The DC coefficient
- ❖ Shifting
- ❖ Conjugate Symmetry
- ❖ Displaying Transforms

Similarity



❖ the forward and inverse transforms are very similar .

❖ The DFT as a spatial filter :

- The following value is independent of f or F

$$\exp \left[\pm 2\pi i \left(\frac{xu}{M} + \frac{yv}{N} \right) \right]$$

- It means the every value of F obtained by multiplying every value of f by a fixed value & adding all the result .
- linear spatial filter multiplies all elements under a mask with fixed values, and adds them all up .

Separability



❖ **The Fourier transform “filter elements” can be expressed as products :**

$$\exp \left[2\pi i \left(\frac{xu}{M} + \frac{yv}{N} \right) \right] = \exp \left[2\pi i \frac{xu}{M} \right] \exp \left[2\pi i \frac{yv}{N} \right].$$

❖ **The first product depends only on x & u, & independent of y & v**

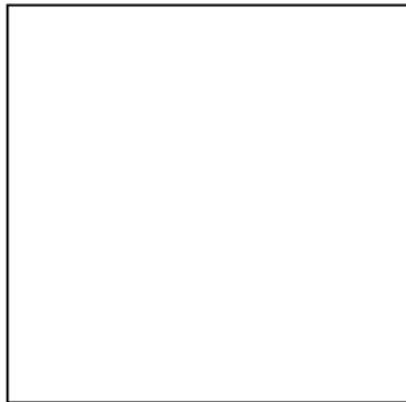
$$\exp \left[2\pi i \frac{xu}{M} \right]$$

❖ **so we can apply it in rows or columns as follow :**

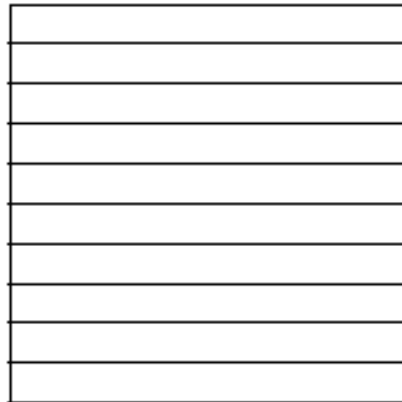
$$F(u) = \sum_{x=0}^{M-1} f(x) \exp \left[-2\pi i \frac{xu}{M} \right],$$

$$f(x) = \frac{1}{M} \sum_{u=0}^{M-1} F(u) \exp \left[2\pi i \frac{xu}{M} \right].$$

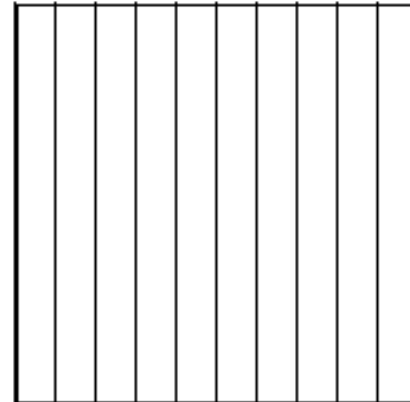
Separability



(a) Original image



(b) DFT of each row of (a)



(c) DFT of each column of (b)

Linearity



- the DFT of a sum is equal to the sum of the individual DFT's :

$$\begin{aligned}\mathcal{F}(f + g) &= \mathcal{F}(f) + \mathcal{F}(g) \\ \mathcal{F}(kf) &= k\mathcal{F}(f)\end{aligned}$$

- It will help us in image degradation such as noise which can be modelled as follow :

$$d = f + n$$

- Where f is the original image , n the noise d degraded image

$$\mathcal{F}(d) = \mathcal{F}(f) + \mathcal{F}(n)$$

- This allow us to remove or reduce n by modifying the transform

The Convolution Method



❖ To calculate $M * S$ using FFT :

1. Pad S with zeroes so that is the same size as M ; denote this padded result by S' .
2. Form the DFT's of both M and S , to obtain $\mathcal{F}(M)$ and $\mathcal{F}(S')$.
3. Form the element-by-element product of these two transforms:

$$\mathcal{F}(M) \cdot \mathcal{F}(S').$$

4. Take the inverse transform of the result:

$$\mathcal{F}^{-1}(\mathcal{F}(M) \cdot \mathcal{F}(S')).$$

Put simply, the convolution theorem states:

$$M * S = \mathcal{F}^{-1}(\mathcal{F}(M) \cdot \mathcal{F}(S'))$$

or equivalently that

$$\mathcal{F}(M * S) = \mathcal{F}(M) \cdot \mathcal{F}(S').$$

The DC coefficient



- ❖ The value $F(0, 0)$ is called the DC coefficient
- ❖ Let $u = v = 0$ then

$$F(0, 0) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) \exp(0) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y).$$

Shifting



- ❖ For purposes of display, it is convenient to have the DC coefficient in the centre of the matrix .
- ❖ This will happen if we multiply the matrix by $(-1)^{x+y}$



<i>A</i>	<i>B</i>
<i>C</i>	<i>D</i>

An FFT

<i>D</i>	<i>C</i>
<i>B</i>	<i>A</i>



After shifting

Figure 4.6: Shifting a DFT

Conjugate Symmetry



- ❖ if we make the substitutions $u = -u$, $v = -v$ then
- ❖ This means that half $\mathcal{F}(u, v) = \mathcal{F}^*(-u + pM, -v + qN)$ a mirror image of the conjugate of the other half.
- ❖ The symmetry means that its information is given in just half of a transform, and the other half is redundant

	a		a^*
b^*	B^*	d^*	A^*
	c		e^*
b	A	d	B

Figure 4.7: Conjugate symmetry in the DFT

Displaying Transforms



- ❖ Elements of $F(u, v)$ are complex numbers to display them we take the magnitude $|F(u, v)|$ this will be of type double .
- ❖ For large ranges we have 2 approaches :
 1. find the maximum value m of $|F(u, v)|$ (this will be the DC coefficient), and use `imshow` to view $|F(u, v)|/m$,
 2. use `mat2gray` to view $|F(u, v)|$ directly.
- ❖ DC coefficient is very much larger than all values this has the effect of showing a transform as a single white dot surrounded by black.
- ❖ To stretch out the values : $\log(1 + |F(u, v)|)$.
- ❖ The display of the magnitude of a Fourier transform is called the **spectrum** of the transform.

Fourier Transform in Matlab



- ❖ **fft** which takes the DFT of a vector.
- ❖ ☐ **ifft** which takes the inverse DFT of a vector.
- ❖ ☐ **fft2** which takes the DFT of a matrix.
- ❖ ☐ **ifft2** which takes the inverse DFT of a matrix.
- ❖ ☐ **fftshift** which shifts a transform.


```
a=ones(8);  
fft2(a)
```

[illegible]

[illegible]

Fourier Transform in Matlab



```
>> a = [zeros(8,4) ones(8,4)]
```

```
a =
```

0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1

```
>> af=fftshift(fft2(a));
```

```
>> round(abs(af))
```

```
ans =
```

0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	9	0	21	32	21	0	9
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

Fourier transform of images



```
>> a=zeros(256,128) ones(256,128)];
```

```
>> af=fftshift(fft2(a));
```

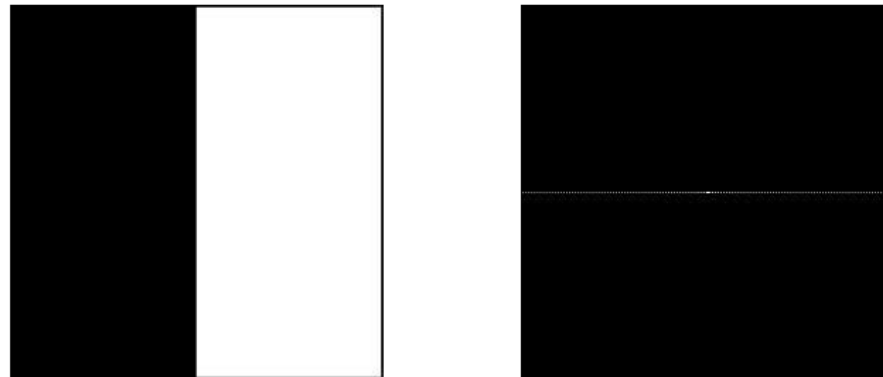


Figure 4.9: A single edge and its DFT

To view the spectrum we have 2 ways :

- `afl=log(1+abs(af)); imshow(afl/afl(129,129))`
- `imshow(mat2gray(log(1+abs(af))))`

Fourier transform of images



```
function fftshow(f,type)

% Usage:  FFTSHOW(F,TYPE)
%
% Displays the fft matrix F using imshow, where TYPE must be one of
% 'abs' or 'log'.  If TYPE='abs', then then abs(f) is displayed; if
% TYPE='log' then log(1+abs(f)) is displayed.  If TYPE is omitted, then
% 'log' is chosen as a default.
%
% Example:
%   c=imread('cameraman.tif');
%   cf=fftshift(fft2(c));
%   fftshow(cf,'abs')
%

if nargin<2,
    type='log';
end

if (type=='log')
    fl = log(1+abs(f));
    fm = max(fl(:));
    imshow(im2uint8(fl/fm))
elseif (type=='abs')
    fa=abs(f);
    fm=max(fa(:));
    imshow(fa/fm)
else
    error('TYPE must be abs or log.');
```

Figure 4.8: A function to display a Fourier transform

Fourier transform of images



Example 2. Now we'll create a box, and then its Fourier transform:

```
>> a=zeros(256,256);  
>> a(78:178,78:178)=1;  
>> imshow(a)  
>> af=fftshift(fft2(a));  
>> figure,fftshow(af,'abs')
```

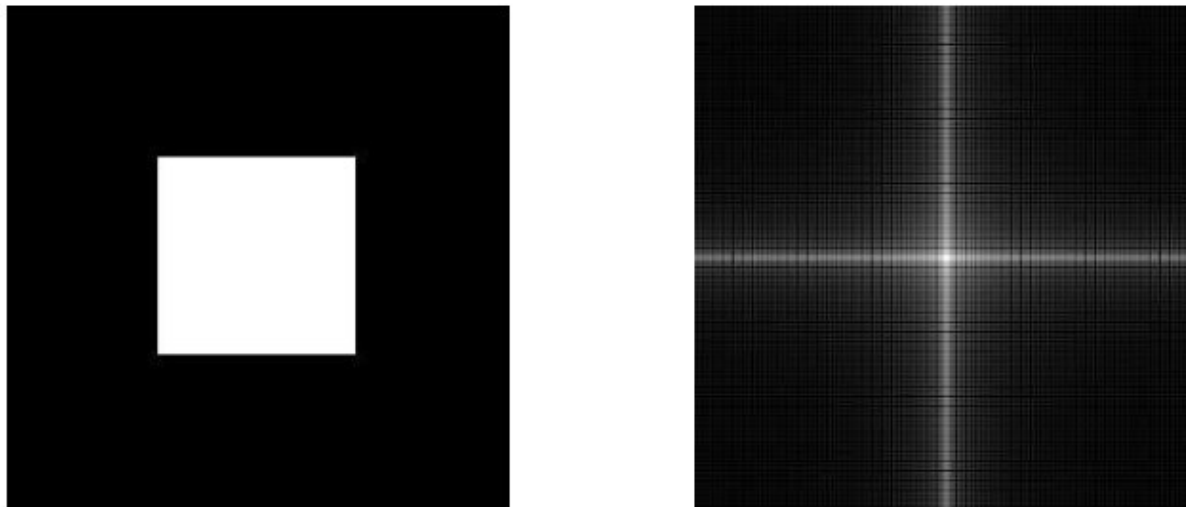


Figure 4.10: A box and its DFT

Fourier transform of images



Example 3. Now we shall look at a box rotated 45° .

```
>> [x,y]=meshgrid(1:256,1:256);  
>> b=(x+y<329)&(x+y>182)&(x-y>-67)&(x-y<73);  
>> imshow(b)  
>> bf=fftshift(fft2(b));  
>> figure,fftshow(bf)
```

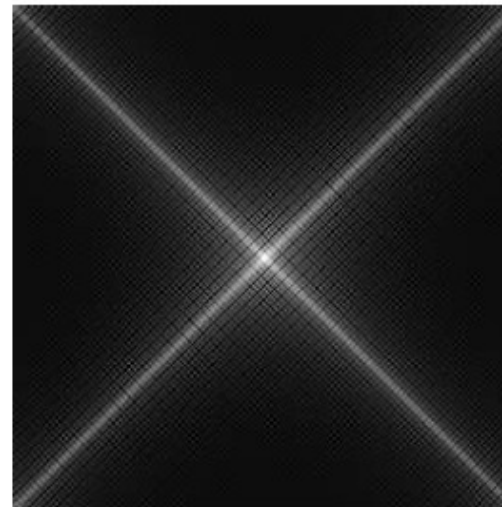
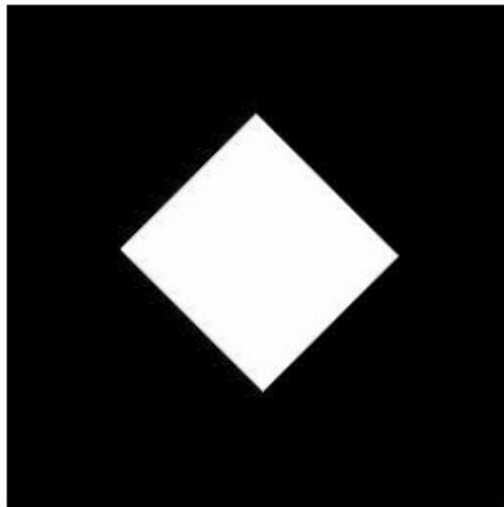


Figure 4.11: A rotated box and its DFT

Fourier transform of images



Example 4. We will create a small circle, and then transform it:

```
>> [x,y]=meshgrid(-128:217,-128:127);  
>> z=sqrt(x.^2+y.^2);  
>> c=(z<15);
```

The result is shown on the left in figure 4.12. Now we will create its Fourier transform and display it:

```
>> cf=fft2shift(fft2(z));  
>> fftshow(cf,'log')
```

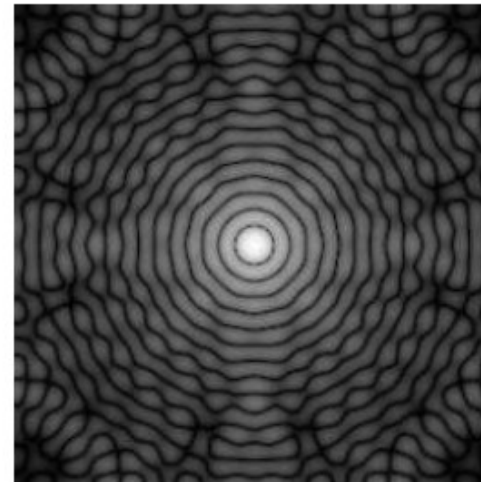
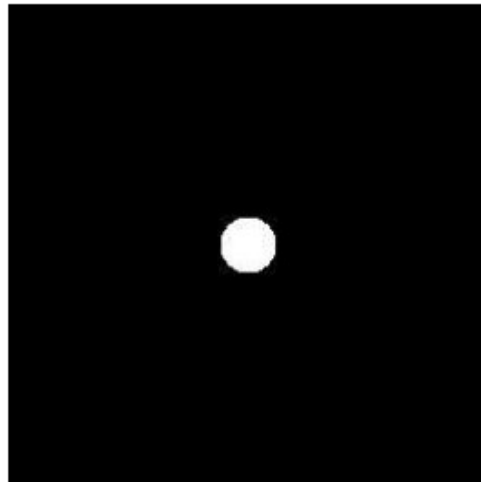


Figure 4.12: A circle and its DFT

Filtering in Frequency Domain



❖ **In this section we will explore 3 methods of filtering :**

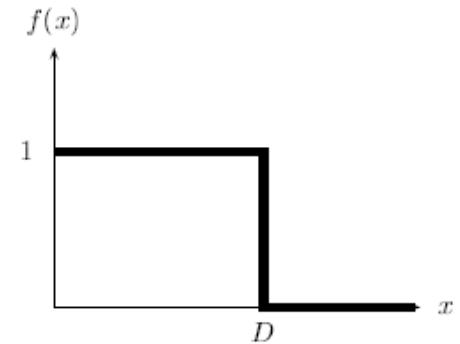
- Ideal filtering
 - Low pass filtering
 - High pass filtering
- Butterworth filtering
- Gaussian filtering

Low pass filtering



$$m(x, y) = \begin{cases} 1 & \text{if } (x, y) \text{ is closer to the center than some value } D, \\ 0 & \text{if } (x, y) \text{ is further from the center than } D. \end{cases}$$

```
>> cm=imread('cameraman.tif');  
>> cf=fftshift(fft2(cm));  
>> figure,fftshow(cf,'log')
```



(a) Low pass

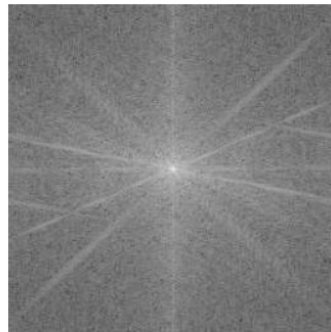


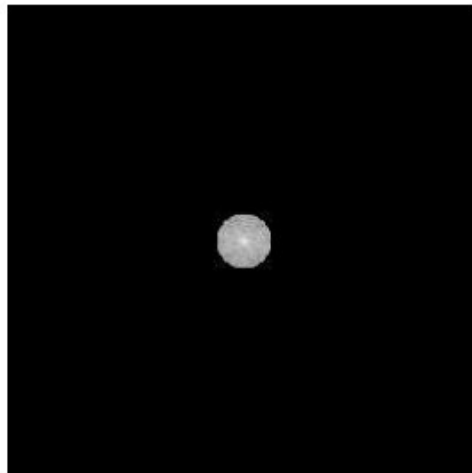
Figure 4.13: The “cameraman” image and its DFT

Low pass filtering

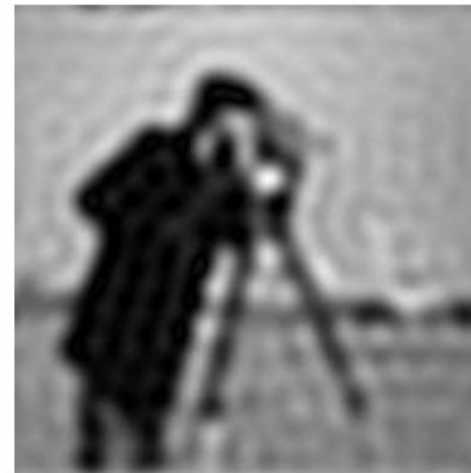


```
>> cfl=cf.*c;  
>> figure,fftshow(cfl,'log')
```

```
>> cfli=ifft2(cfl);  
>> figure,fftshow(cfli,'abs')
```



(a) Ideal filtering on the DFT

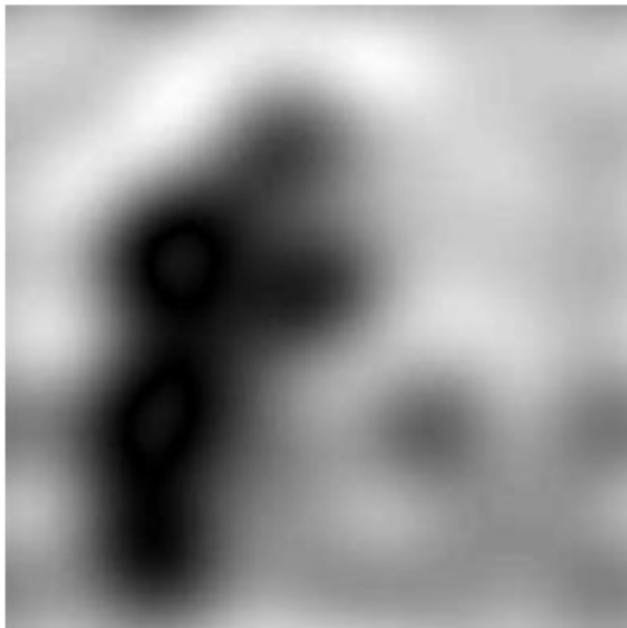


(b) After inversion

Figure 4.14: Applying ideal low pass filtering

Note :dot asterisk is the Matlab syntax for element-wise multiplication of two matrices

Ideal low pass filtering with different cutoffs



(a) Cutoff of 5



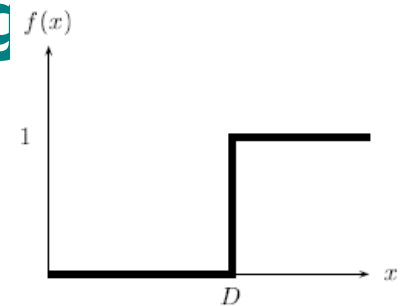
(b) Cutoff of 30

Figure 4.15: Ideal low pass filtering with different cutoffs

High Pass filtering



❖ **The High pass filtering is the opposite of low pass filtering**



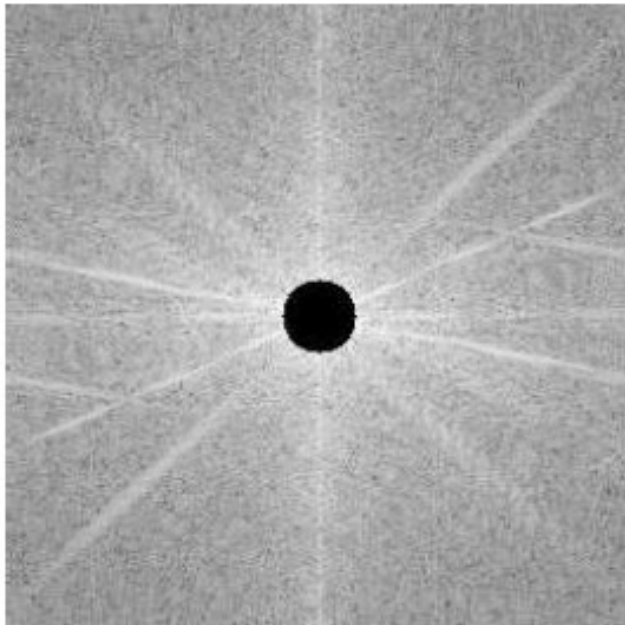
(b) High pass

```
>> [x,y]=meshgrid(-128:127,-128:127);  
>> z=sqrt(x.^2+y.^2);  
>> c=(z>15);
```

```
>> cfh=cf.*c;  
>> figure,fftshow(cfh,'log')
```

```
>> cfhi=ifft2(cfh);  
>> figure,fftshow(cfhi,'abs')
```

High Pass filtering



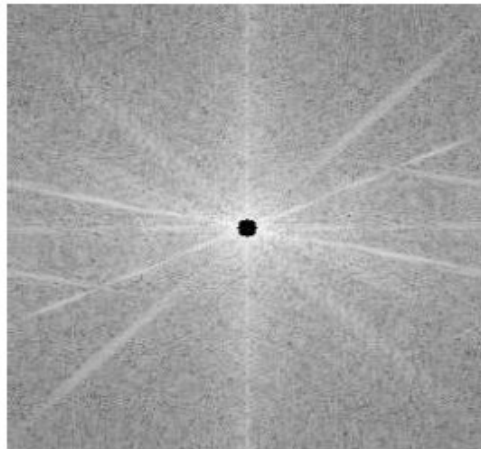
(a) The DFT after high pass filtering



(b) The resulting image

Figure 4.16: Applying an ideal high pass filter to an image

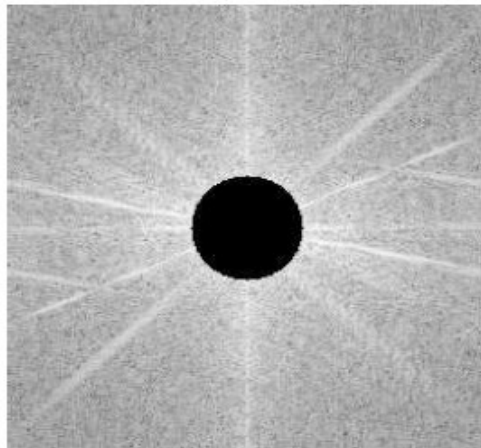
Ideal ^{high} pass filtering with different cutoffs



(a) Cutoff of 5



(b) The resulting image



(a) Cutoff of 30



(b) The resulting image

Figure 4.17: Ideal high pass filtering with different cutoffs

Butterworth filtering



❖ Ideal Filtering Disadvantage:

- Cutoff the Fourier transform at some distance from the center introducing unwanted artifacts ranging into the result .
- We use a filter matrix with less sharp cut off (Butterworth filtering).

Butterworth filtering



❖ Low pass filter

$$f(x) = \frac{1}{1 + (x/D)^{2n}}$$

❖ High pass filter

$$f(x) = \frac{1}{1 + (D/x)^{2n}}$$

Note : n is called the order of the filter the size of n dictates the sharpness of the cutoff

Butterworth filtering

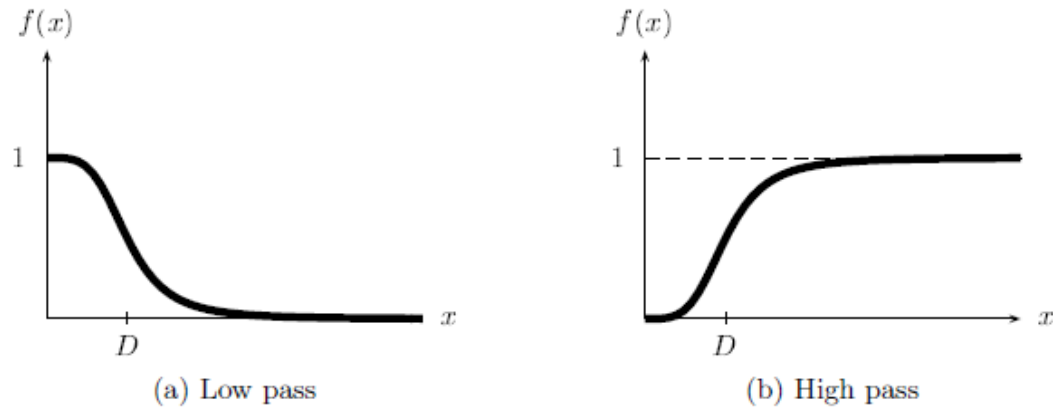


Figure 4.19: Butterworth filter functions with $n = 2$

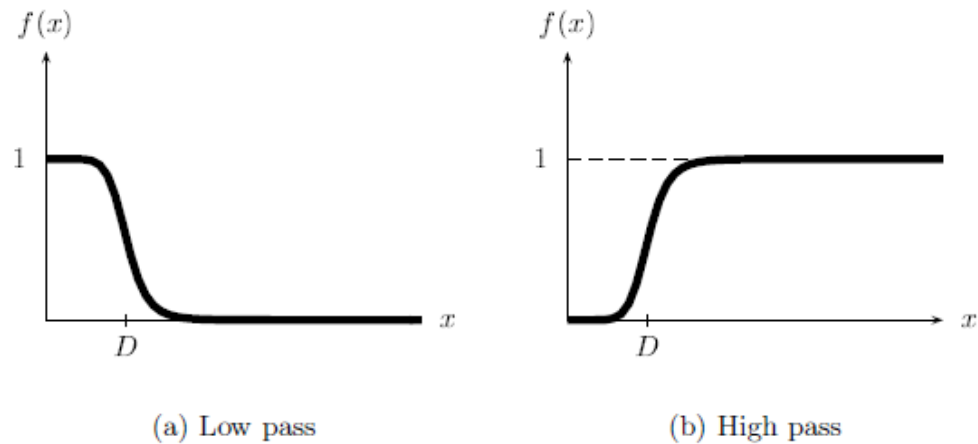


Figure 4.20: Butterworth filter functions with $n = 4$

Low pass Butterworth function



```
function out=lbutter(im,d,n)
% LBUTTER(IM,D,N) creates a low-pass Butterworth filter
% of the same size as image IM, with cutoff D, and order N
%
% Use:
%   x=imread('cameraman.tif');
%   l=lbutter(x,25,2);
%
height=size(im,1);
width=size(im,2);
[x,y]=meshgrid(-floor(width/2):floor((width-1)/2),-floor(height/2): ...
               floor((height-1)/2));
out=1./(1+(sqrt(2)-1)*((x.^2+y.^2)/d^2).^n);
```

Figure 4.21: A function to generate a low pass Butterworth filter

High pass Butterworth function



```
function out=hbutter(im,d,n)
% HBUTTER(IM,D,N) creates a high-pass Butterworth filter
% of the same size as image IM, with cutoff D, and order N
%
% Use:
%   x=imread('cameraman.tif');
%   l=hbutter(x,25,2);
%
out=1-lbutter(im,d,n);
```

Figure 4.22: A function to generate a high pass Butterworth filter

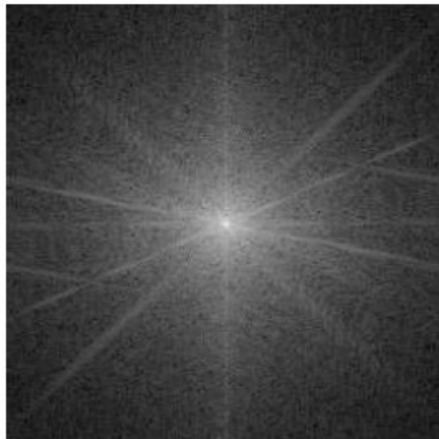
Note : A butterworth high bass filter can be obtained by subtracting a low pass filter from 1

Butterworth filtering



❖ To produce a butterworth filter of size 256×256 with $D = 15$ & order $n = 2$

```
>> [x,y]=meshgrid(-128:217,-128:127));  
>> bl=1./(1+((x.^2+y.^2)/15).^2);  
  
>> bl=lbutter(c,15,1);  
>> cfbl=cf.*bl;  
>> figure,fftshow(cfbl,'log')
```



(a) The DFT after Butterworth low pass filtering

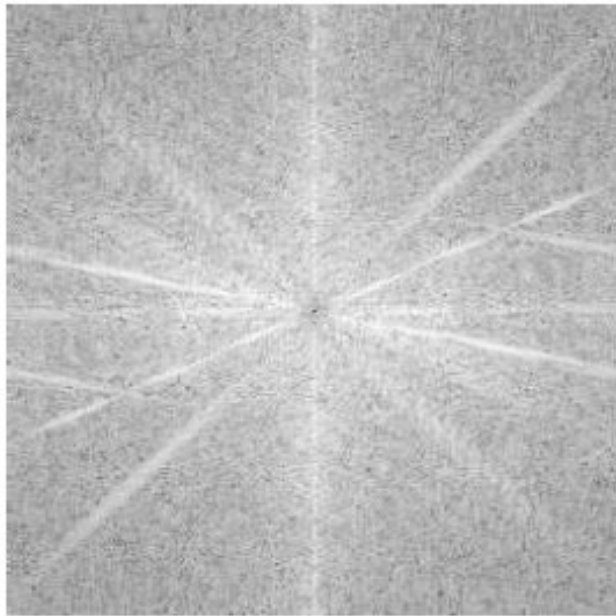


(b) The resulting image

Butterworth filtering



```
>> bh=hbutter(cm,15,1);  
>> cfbh=cf.*bh;  
>> figure,fftshow(cfbh,'log')  
  
>> cfbhi=ifft2(cfbh);  
>> figure,fftshow(cfbhi,'abs')
```



(a) The DFT after Butterworth high pass filtering



(b) The resulting image

Figure 4.24: Butterworth high pass filtering

Gaussian filtering



❖ **Gaussian filters may be considered to be the most smooth of all the filters.**

```
>> g1=mat2gray(fspecial('gaussian',256,10));  
>> cg1=cf.*g1;  
>> fftshow(cg1,'log')  
>> g2=mat2gray(fspecial('gaussian',256,30));  
>> cg2=cf.*g2;  
>> figure,fftshow(cg2,'log')
```

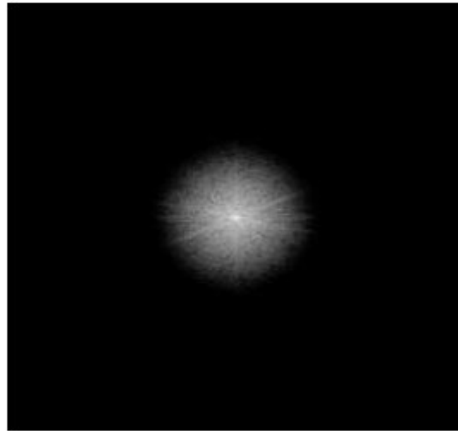
```
>> g=fspecial('gaussian',256,10);  
>> format long, max(g(:)), format
```

ans =

0.00158757552679

```
>> cgi1=ifft2(cg1);  
>> cgi2=ifft2(cg2);  
>> fftshow(cgi1,'abs');  
>> fftshow(cgi2,'abs');
```

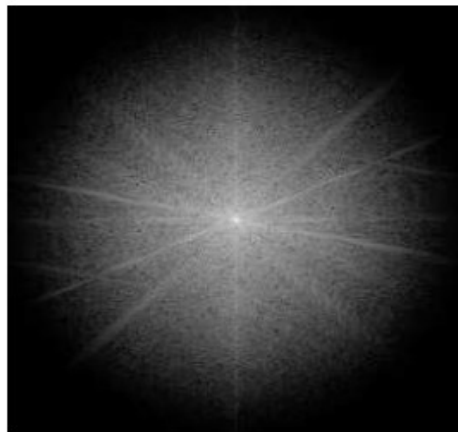
Gaussian filtering



(a) $\sigma = 10$



(b) Resulting image



(c) $\sigma = 30$



(d) Resulting image

Figure 4.25: Applying a Gaussian low pass filter in the frequency domain

High pass Gaussian filtering



```
>> h1=1-g1;  
>> h2=1-g2;  
  
>> ch1=cf.*h1;  
>> ch2=cf.*h2;  
>> ch1i=ifft2(ch1);  
>> chi1=ifft2(ch1);  
>> chi2=ifft2(ch2);  
>> fftshow(chi1,'abs')  
>> figure,fftshow(chi2,'abs')
```

High pass Gaussian filtering



(a) Using $\sigma = 10$



(b) Using $\sigma = 30$

Figure 4.26: Applying a Gaussian high pass filter in the frequency domain



Thank You